

Neural-Inspired Hash Function

RIYADI ADNANE

October 30, 2024

—

Blockchain

—

Pr. Ikram Ben Abdel Ouahab

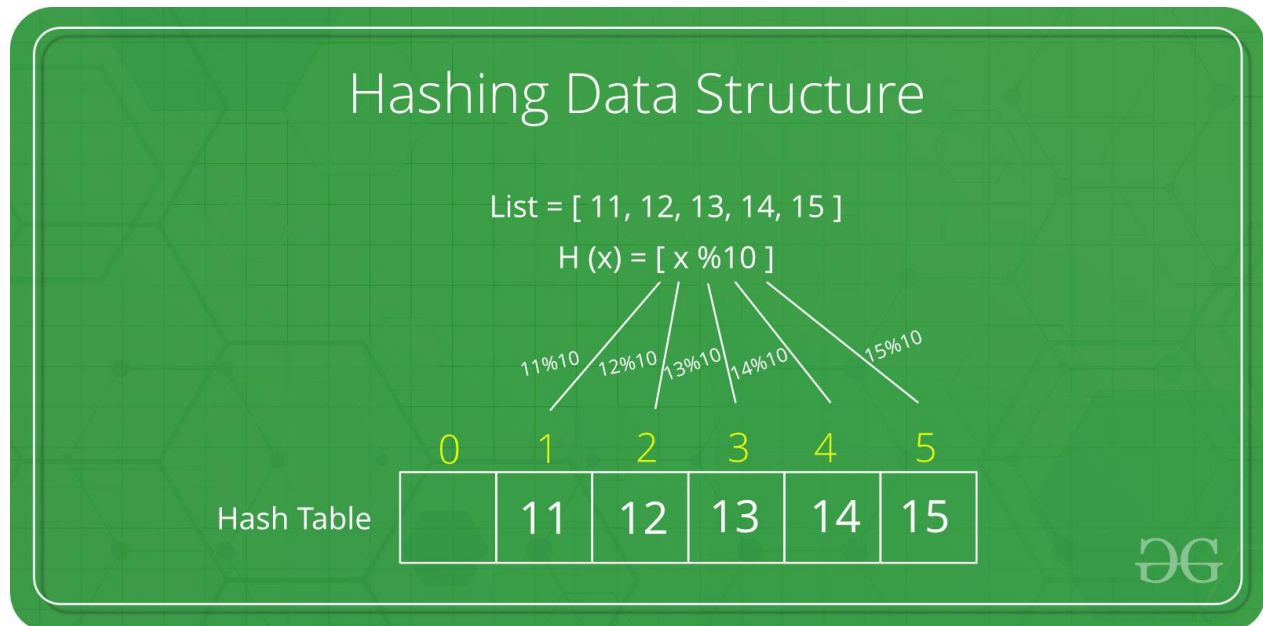
Table of Contents

Understand Hashing.....	3
What is Hashing?.....	3
What is a Hash function - or a hashing Algorithm?	3
Key Properties of Hashing.....	3
Overview of the Neural-Inspired Hash Function	4
Architecture	4
Components	4
Default Configuration	4
Process Flow.....	5
Input Processing.....	5
Neural Network Processing.....	5
Hash Generation.....	5
Breakdown of the code:.....	5
1. NeuralHash Class.....	5
2. NeuralLayer Class.....	6
3. HashTester Class.....	6
4. Main Execution Block	7

Understand Hashing

What is Hashing?

Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the storage of an item in a data structure.



What is a Hash function - or a hashing Algorithm?

The **hash function** creates a mapping between key and value, this is done through the use of mathematical formulas known as hash Algorithms. The result of the hash function is referred to as a hash value or hash. The hash value is a representation of the original string of characters but usually smaller than the original.

Hashes are the output of a hashing algorithm like MD5 (Message Digest 5) or SHA (Secure Hash Algorithm). These algorithms essentially aim to produce a unique, fixed-length string – the hash value, or “message digest” – for any given piece of data or “message”. As every file on a computer is, ultimately, just data that can be represented in binary form, a hashing algorithm can take that data and run a complex calculation on it and output a fixed-length string as the result of the calculation. The result is the file’s hash value or message digest.

Key Properties of Hashing

1. **Fixed-Length Output:** Regardless of the input size, a hash function produces an output of a consistent, fixed length. For example, SHA-256 always produces a 256-bit hash, even if the input is a single character or an entire book.
2. **Deterministic:** A hash function will always generate the same hash value for the same input, ensuring consistency and reliability in applications like data verification.

3. **Fast Computation:** Hash functions are designed to be computationally efficient, allowing for quick hash generation, even for large inputs.
4. **Pre-Image Resistance:** Given a hash value, it should be practically impossible to reverse-engineer the original input, ensuring security for sensitive data.
5. **Avalanche Effect:** Small changes in input (even a single character) result in a drastically different hash. This is crucial in ensuring that similar inputs don't yield similar outputs, preventing information leakage.
6. **Collision Resistance:** A hash function should make it highly unlikely for two distinct inputs to produce the same output, a property called a "collision." High-quality hash functions are designed to minimize these collisions.

Overview of the Neural-Inspired Hash Function

The Neural-Inspired Hash Function is a novel approach to cryptographic hashing that combines neural network principles with traditional hashing techniques. It creates a fixed-size hash output from variable-length input text using a multi-layer neural network architecture.

Architecture

Components

1. **NeuralLayer Class**
 - Handles individual neural network layers
 - Implements weights, biases, and ReLU activation
 - Performs forward propagation through the layer
2. **NeuralHash Class**
 - Main hash function implementation
 - Manages the neural network architecture
 - Handles input preprocessing and final hash generation
3. **HashTester Class**
 - Provides testing utilities for hash function properties
 - Tests determinism, uniqueness, fixed length output, and speed

Default Configuration

- Input size: 256 neurons (255 for input + 1 for salt)
- Hidden layers: [128, 64, 32] neurons
- Output size: 16 neurons
- Final hash length: 32 characters (128 bits)

Process Flow

Input Processing

1. Text input is converted to ASCII values
2. Values are padded/truncated to match input size
3. Salt is added for additional security
4. Input is normalized to range [0, 1]

Neural Network Processing

1. Forward propagation through multiple layers
2. ReLU activation between layers
3. Final layer produces fixed-size output

Hash Generation

1. Neural network output is transformed
2. XOR operation with salt-based mask
3. Final SHA-256 transformation
4. Truncation to 32 characters

Breakdown of the code:

1. NeuralHash Class

The NeuralHash class is the core of the hashing algorithm, designed to convert a text input into a secure, deterministic hash through a neural network and additional transformations.

- **__init__ Method:** Initializes the NeuralHash instance.
 - **Parameters:**
 - **input_size:** The size of the input vector for the neural network, minus one spot reserved for the salt.
 - **hidden_sizes:** Defines the sizes of the hidden layers in the network.
 - **output_size:** Size of the final layer, which is ultimately hashed to produce the final result.
 - **seed:** Seed for random number generation, ensuring consistent behavior.
 - **Layers:** Creates a series of NeuralLayer instances based on the specified architecture, with each layer initialized using a different seed.
 - **Salt Generation:** Adds a unique integer salt to increase hash security.
- **preprocess_input Method:** Converts input text into a numeric vector for input into the neural network.
 - **Conversion:** Translates each character in the input text to an ASCII value.
 - **Padding/Truncation:** Adjusts the input vector to match the network's input_size.
 - **Salt Addition:** Appends the salt as the final element in the input vector.

- **Normalization:** Normalizes the ASCII values to be within $[0, 1]$ by dividing by 255.
- **forward_pass Method:** Processes the input vector through the neural network layers.
 - **Processing:** Each layer applies a weight and bias, followed by the ReLU activation function to produce the output of that layer.
- **final_transform Method:** Applies an XOR-based transformation on the output to enhance security and ensure irreversibility.
 - **Conversion to Bytes:** Converts the output array to bytes.
 - **Mask Generation:** Uses SHA-256 of the salt to create a mask, ensuring uniqueness and additional security.
 - **XOR Masking:** Applies an XOR operation with the mask for the final transformation.
- **hash Method:** Generates the hash from the text input.
 - **Process:**
 - Preprocesses the input using preprocess_input.
 - Passes the input through the neural layers with forward_pass.
 - Applies final_transform for security.
 - Finally, computes a SHA-256 hash of the transformed output, returning the first 32 characters.

2. NeuralLayer Class

Each instance of NeuralLayer represents a single layer in the neural network, performing a linear transformation followed by an activation.

- **__init__ Method:** Initializes the layer with random weights and biases.
 - **Weights:** Initialized as small random values (scaled by 0.01) for stability.
 - **Biases:** Also initialized as small random values.
- **forward Method:** Executes a forward pass through the layer.
 - **Weighted Sum:** Computes the weighted sum of the inputs and adds biases.
 - **Activation:** Applies the ReLU function, setting all negative values to zero.

3. HashTester Class

This class provides test functions to validate the properties of the NeuralHash algorithm, ensuring reliability and efficiency.

- **test_determinism Method:** Verifies that hashing the same input multiple times produces identical results.
 - **Returns** True if the hash is consistent, otherwise False.
- **test_uniqueness Method:** Confirms that different inputs produce different hashes.
 - **Returns** True if all hashes are unique, otherwise False.
- **test_fixed_length Method:** Ensures that the hash output is a consistent length.
 - **Returns** True if the length is consistent, otherwise False.

- **test_speed Method:** Measures the average computation time for generating a hash.
 - **Returns** the average time in seconds per hash computation.

4. Main Execution Block

The main execution block initializes an instance of NeuralHash and runs each test on sample inputs, printing the test results and example hashes.